api0

GUIDE 04

api0 Architecture and Supported Authentication Models

The technical guide: how a request travels through api0, from Claude, Teams or WhatsApp to your application, and every inbound and outbound authentication model, with when to choose which.

Updated 7 November 2026 · Read online: api0.ai/guides/architecture-and-auth

---

This guide is for technical and security teams evaluating api0. It describes the path of a request, then the two authentication questions every AI gateway has to answer: **who is calling**, and **which credential your application is called with**.

Overview



Three components, written in **Rust**:

- **The gateway** receives every call, authenticates it, routes it and logs it. It's the only path to your systems.
- **The store** holds the tool registry, tenants, credentials (encrypted) and the usage ledger.
- **The messaging bridge** receives webhooks from Teams, WhatsApp and Telegram and runs the assistant, which calls the same tools through the gateway.

The MCP gateway

`/mcp` is a standard **Model Context Protocol** server over HTTP (Streamable HTTP). Nothing proprietary at the protocol level: any MCP client works.

Method	Behaviour
`initialize`	Negotiates the protocol version, advertises tools, returns the tenant's instructions
`tools/list`	The caller's tenant's tools, and only those
`tools/call`	Routes to the application, meters, returns the result
`ping`	Empty result

Supported protocol versions: `2025-06-18`, `2025-03-26`, `2024-11-05`.

Tenant instructions. `initialize` returns a text (4,000 characters max) that the client places in front of the model: shared usage rules (load a tool before calling it, take IDs from a listing tool rather than guessing…), the tenant's own text (2,000 characters max) and, for each tool group, its starting tools.

Errors. A refusal from your application comes back as a **result** flagged `isError`, with your application's own message: the model reads it and can correct. JSON-RPC errors are reserved for api0's own refusals (unknown tool, invalid parameters, no credits). **A failed call is never billed.**

What a tool is

A tool is an entry in the registry, scoped to one tenant:

Field	Role
name, description, input schema	What the model sees
application URL	Target, with `{parameters}` substituted
HTTP verb	Set → REST call. Empty → the application speaks MCP natively
content type, body template	For APIs whose body isn't the arguments object (JSON Patch…)
static headers	The tool's own constants
identity forwarding	Whether to send the user's email and tenant
cost, timeout	Metering and patience

Importing an OpenAPI spec creates these entries automatically. **Body templates** cover the hard cases without code: an optional field that wasn't supplied disappears from the body (a whole JSON Patch operation in an array, a single key in an object). **Parameter names** MCP clients reject (`$filter`, `page[size]`) are exposed under a safe name and sent to the application under their real name.

Header order

Applied in this order, last wins:

1. **api0 identity** (`X-User-Email`, `X-Tenant-Id`) — only if identity forwarding is on;

2. the tool's **static headers**;
3. **outbound authentication** — always wins a name collision;
4. `Content-Type`.

The gateway never sends an internal secret to an application.

Inbound authentication: who is calling?

Model	Used by	Detail
OAuth 2.1 + PKCE	Claude and MCP clients	The client registers with the tenant's Client ID; the user signs in; the gateway issues a token bound to that user and that tenant
Google sign-in	Default	For the dashboard and user sign-in
Your identity provider (OIDC)	Enterprises	Entra ID or any OIDC IdP configured per tenant: your employees sign in with their account, MFA and conditional access included. The IdP client secret is encrypted
API key	Server-to-server integrations	One key per tenant or per consumer
Messaging account linking	Teams, WhatsApp, Telegram	On the first message, the user gets a short-lived link to tie their messaging identity to their account

Acting for an end user

An application can call the gateway with its key **on behalf of one of its users**, by naming them. The gateway checks this is allowed before any call: the key's owner, always; a consumer key, never for anyone else; a tenant key, only for someone who joined that tenant themselves (through OAuth consent or their own key). Being added as a member is not enough. Otherwise: `403`.

Outbound authentication: which credential calls your application?

Each tenant has a **default connection**, and can add named ones when some tools target a service that needs a different sign-in.

Mode	What the application receives	When to choose it
`none`	Nothing	Public API
api0 identity token (`google_sa` , no key)	A Google identity token signed by api0's service account	**Your own applications.** No secret to store
`google_sa` with your key	An identity token signed by **your** service account	Google Cloud applications (Cloud Run…) with your own IAM
`static_bearer`	`Authorization: Bearer <token>`	Third-party API with a single key

Mode	What the application receives	When to choose it
`header_injection`	The headers you define	APIs with proprietary headers
`client_credentials`	A token obtained via OAuth client credentials	APIs that issue service tokens
`per_user` — personal secret	**Each** user's own credential: `Basic` (Azure DevOps PAT), `Bearer`, or a raw header	Per-person attribution and permissions
`per_user` — OAuth	**Each** user's OAuth token, refreshed by api0	Systems with OAuth sign-in (e.g. Microsoft)

The api0 identity token in detail

For an application you run, the simplest option is often to store **no** secret at all: the gateway signs every call with api0's service account.

One signer for every tenant means the **audience** is what separates them, and the tenant doesn't choose it. The gateway builds it from the application URL's origin and the calling tenant's ID:

```
https://api.example.com/orders  →  aud = https://api.example.com/api0/tenant/<tenant id>
```

Only the origin is kept: no path or query can spell out another tenant's audience. Your application checks two values, once:

Check	Value
`aud`	`https://<your origin>/api0/tenant/<your tenant>`
`email` (signer)	api0's service account

Pinning the signer is mandatory: any Google service account can mint a token for any audience.

Per-person credentials

This is the model to prefer whenever attribution matters.

- **Personal secret.** The user adds their token once (an Azure DevOps PAT, for example). It's encrypted and **can't be read back**, only replaced. An optional verification URL can check who the token belongs to.
- **OAuth.** The user connects their account once. api0 keeps the refresh token encrypted and renews access. The OAuth flow's `state` value is single-use and short-lived (CSRF protection).
- **No credential yet**: the user gets a tool result explaining how to add one. **Never** a call made as someone else.

Result: the target system sees the real person, applies their permissions and attributes the action to them.

Protecting secrets

- **AES-256-GCM** encryption, sealed to the tenant and the secret's purpose: a secret copied into another context won't open.
- Secrets are never returned to the MCP client, the model or the dashboard (metadata only).
- With no encryption key configured, the store **refuses** to save a credential rather than keep it in plain text.

Personal data

Identity forwarding (`X-User-Email`, `X-Tenant-Id`) is a **per-tool** setting. It says who the call is for, not that it came from api0: an application that needs to know that verifies the outbound credential (the signed token). **Turn it off for any tool pointed at a third party**: your users' emails stay inside the platform.

Usage, credits and logs

- Every call is logged: tool, user, tenant, status, latency.
- Every successful call uses credits; a failure is never billed.
- Billing mode per tenant: **the tenant pays**, or **each person pays** on their own plan.

Choosing a model: the decision tree

1. **Is the application yours?** → api0 identity token. Nothing to store.
2. **Must actions be attributed to the person, or their permissions applied?** → `per_user` (OAuth if the system supports it, otherwise a personal secret).
3. **Third-party API with a service key?** → `static_bearer`, `header_injection` or `client_credentials`, with identity forwarding off.
4. **Tools targeting different services?** → one named connection per service.